

DSPO: STABLE AND EFFICIENT POLICY OPTIMIZATION FOR AGENTIC SEARCH AND REASONING

Anonymous authors
Anonymous submission

ABSTRACT

Enhancing LLMs with the ability to actively search external knowledge is crucial for complex and real-world tasks, yet current approaches either lean on prompting to elicit innate agent behavior or hit performance ceilings and collapse when RL is applied to complex interactive tasks, leaving their agentic potential untapped. This paper makes the following contributions. **(i)** We introduce **Dynamic-filter Sequence-level Policy Optimization (DSPO)**, an improved RL algorithm that delivers robust agent training by unifying sequence-level optimization with dynamic sample filtering, directly targeting the instability and sample inefficiency that break RL on search tasks. **(ii)** We show that DSPO trains agents purely through RL to interleave multi-turn search and reasoning, obviating the need for supervised demonstration data. **(iii)** We demonstrate strong, reproducible empirical gains: across multiple QA benchmarks, our 7B model improves over a comparable previous work by **34.1%**, and even outperforms the 14B model from previous work on complex multihop QA such as HotpotQA by nearly **9% relative**, all while maintaining exceptional training stability. Together, these contributions establish DSPO as a stable and efficient recipe for policy optimization in agentic search and reasoning.

1 INTRODUCTION

Large Language Models (LLMs) (Brown et al., 2020; Touvron et al., 2023; Zhao et al., 2023) have demonstrated exceptional performance across a spectrum of specialized tasks, including math (Shao et al., 2024; Trinh et al., 2024; Yu et al., 2025), coding (Zheng et al., 2023; Yang et al., 2024), and creative writing (Chakrabarty et al., 2024; Marco et al., 2024). However, a fundamental limitation persists: their knowledge is inherently static, confined to the data on which they were trained. The dominant remedy to this knowledge cutoff is to equip LLMs with search capabilities, transforming them into agents that can actively query external knowledge sources (Jin et al., 2025b). This ability is a prime example of tool-calling (Schick et al., 2023), where the model learns to interact with an external search tool to solve problems it cannot answer alone. Crucially, mastering this skill requires learning a complex, multi-step policy, which frames the task as a sequential decision-making problem ideal for Reinforcement Learning (RL)—and it is precisely the difficulty of learning this policy stably that motivates the algorithm we contribute in this work.

Unlike Supervised Fine-Tuning (SFT), which relies on costly static demonstrations and fails to teach exploration, RL provides a framework for LLMs to learn effective policies through trial-and-error (Chu et al., 2025). Consequently, value-free methods like Group Relative Policy Optimization (GRPO) (Shao et al., 2024) have become a dominant paradigm, prized for their simplicity and reduced memory overhead. Yet the central obstacle this paper addresses is that, despite GRPO’s success in more constrained tasks, applying it to the open-ended domain of interactive search reveals critical instabilities (Jin et al., 2025b; Yu et al., 2025; Cui et al., 2025; Liu et al., 2025). We trace this fragility to two fundamental flaws that jointly define the problem our contribution must solve. First, as identified by Zheng et al. (2025), GRPO’s token-level objective is ill-posed when paired with a sequence-level reward, creating high-variance gradients that destabilize training. Second, the sparse rewards inherent to search tasks often yield sample groups with homogeneous outcomes (e.g., all successes or all failures), causing the advantage signal to collapse and providing no learning signal, which severely hinders sample efficiency (Yu et al., 2025; Liu et al., 2025). These two failure modes—unstable gradients and vanishing learning signal—are the precise targets of DSPO.

Our central contribution addresses these two challenges directly: we introduce **Dynamic-filter Sequence-level Policy Optimization (DSPO)**, an algorithm that synthesizes and refines key principles from recent policy optimization research into one coherent training recipe. To fix the first failure mode, DSPO adopts sequence-level optimization from GSPO (Zheng et al., 2025) to match the unit of optimization with the unit of reward; aligning the objective with the reward signal fundamentally stabilizes learning for long-horizon reasoning tasks. To fix the second, DSPO incorporates

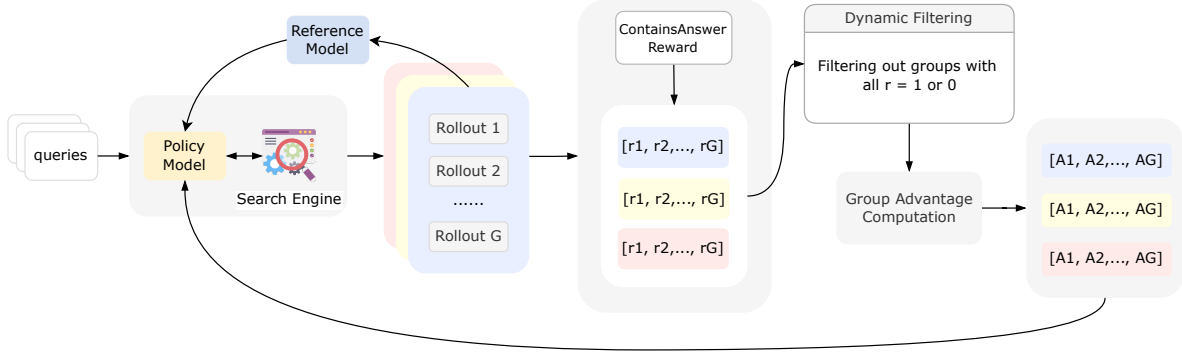


Figure 1: An overview of the DSPO training loop, which visualizes the two contributed components working together. For a given query, the policy model generates a group of G trajectories by interacting with the search environment, and each trajectory is assigned a sparse terminal reward. Our first component, the **dynamic filter**, discards groups with homogeneous outcomes and keeps sampling until a batch is filled, ensuring that every training batch provides an effective advantage signal. Advantages are then computed and used to update the policy through our second component, the **sequence-level objective**.

a dynamic outcome-based filtering mechanism inspired by DAPO (Yu et al., 2025) that actively constructs training batches from rollout groups containing both successful and unsuccessful outcomes for each prompt, guaranteeing that the advantage signal $\hat{A}_i$ stays effective and stable. The contribution is not merely the reuse of these ideas but their integration: by combining the two components into a single framework, DSPO becomes a stable and high-performance algorithm purpose-built for complex, multi-turn search and reasoning. The payoff is concrete—our model achieves a **34.1% relative improvement** over a leading 7B baseline (Jin et al., 2025b) and even surpasses its 14B counterpart (Jin et al., 2025a) on complex multi-hop benchmarks like HotpotQA, outperforming it by **nearly 9% relative** (0.613 vs. 0.563).

In summary, this paper makes three main contributions, which we signpost here and revisit throughout the paper:

- **Contribution 1 (Algorithm).** We propose DSPO, an improved RL algorithm that overcomes the core instability and sample-inefficiency issues in training agentic search models. It achieves this by unifying two key principles into a single cohesive framework: **sequence-level optimization** for robust policy updates and **dynamic outcome-based filtering** for a dense and effective learning signal.
- **Contribution 2 (Performance).** We demonstrate DSPO’s substantial performance gains through rigorous benchmarking. Our 7B model achieves a **34.1% relative improvement** over a comparable 7B baseline and, more strikingly, outperforms its 14B counterpart on complex multi-hop QA, achieving a **nearly 9% relative gain** on HotpotQA (0.613 vs. 0.563).
- **Contribution 3 (Stability under a controlled setup).** We provide extensive empirical evidence for DSPO’s superior training stability, showing it enables a smooth, monotonic learning trajectory where baselines collapse. Crucially, these results are obtained using only a basic BM25 retriever, isolating the gains to the robustness of our algorithm rather than to a stronger retrieval stack.

2 RELATED WORK

2.1 RL FOR LLMs

The landscape of RL for LLMs has evolved rapidly, moving from foundational Reinforcement Learning from Human Feedback (RLHF) methods that use PPO and explicit reward models (Ouyang et al., 2022; Christiano et al., 2017; Schulman et al., 2017) to simpler, direct-optimization frameworks like DPO (Rafailov et al., 2023). A key shift towards value-free optimization is marked by Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which simplifies training by deriving a reward signal from group statistics. However, GRPO’s token-level objective is known to cause training instability (Liu et al., 2025; Cui et al., 2025), prompting several targeted improvements: GSPO shifts to a sequence-level objective to match the unit of reward (Zheng et al., 2025), DAPO tackles inefficient learning from sparse rewards with a dynamic outcome-based sampling mechanism (Yu et al., 2025), and GMPO stabilizes the token-

level objective using a geometric-mean aggregation to reduce sensitivity to outliers (Zhao et al., 2025). The gap our contribution fills is that, despite these advances, each of these algorithms individually still faces training collapse or performance bottlenecks in our agentic-search experiments. This positions DSPO’s contribution precisely: rather than proposing yet another isolated fix, we synthesize sequence-level optimization and dynamic filtering into one unified algorithm that jointly overcomes the two failure modes unique to training autonomous search agents.

2.2 LLMs WITH AGENTIC RETRIEVAL

To mitigate the static knowledge limitations of LLMs, RAG integrates external retrievers to dynamically incorporate evolving information (Lewis et al., 2020; Gao et al., 2023). Classic RAG frameworks employ dense retrievers to fetch relevant documents, which are then concatenated into the LLM’s input for generation (Karpukhin et al., 2020). However, these approaches often rely on fixed pipelines, limiting autonomy in complex, multi-turn scenarios. Research has since evolved toward agentic paradigms, where LLMs act as autonomous agents capable of planning, searching, and reasoning iteratively. Frameworks like ReAct synergize reasoning and acting, enabling LLMs to interact with tools for tasks such as web navigation (Yao et al., 2023), while multi-agent systems, including AutoGen, facilitate collaborative workflows (Wu et al., 2024). More recent work emphasizes agentic RAG and RL integration, where agents enhance retrieval through decision-making: Wu et al. (2025) introduce Agentic Reasoning, a framework integrating external tools for streamlined LLM reasoning, while RL-integrated approaches (Jin et al., 2025b; Chen et al., 2025; Song et al., 2025) train LLMs to interleave reasoning and search purely through RL. This end-to-end paradigm internalizes agent capabilities and avoids the engineering overhead of multi-agent frameworks, but these methods still grapple with training instability and performance limitations in the open-ended search domain. It is exactly this bottleneck that our contribution targets: DSPO provides a robust and efficient training framework that ensures stable policy optimization, enabling LLMs to learn effective multi-turn search strategies where prior end-to-end methods stall.

3 METHODOLOGY

This section develops our first contribution—the DSPO algorithm—in three steps that mirror its logic. We first formulate agentic search as an RL problem and review prior policy optimization algorithms, making explicit the two limitations DSPO is designed to remove. We then introduce **D**ynamic-filter **S**equence-level **P**olicy **O**ptimization (DSPO), presenting its two core components—sequence-level optimization for stability and dynamic filtering for efficiency—as direct answers to those two limitations. Finally, we show how the components combine into a single integrated training algorithm.

3.1 PRELIMINARIES

Policy Gradient Methods for LLMs. Training LLMs via RL often employs policy gradient methods like PPO (Schulman et al., 2017), a popular algorithm for LLM alignment. It optimizes a policy π_θ by maximizing a clipped surrogate objective using samples from an old policy $\pi_{\theta_{\text{old}}}$. The objective, averaged over tokens, is given by:

$$J_{\text{PPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{|y|} \sum_{t=1}^{|y|} \min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right], \quad (1)$$

where $r_t(\theta) = \frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})}$ is the token-level importance ratio. A key drawback, which motivates the value-free line of work DSPO builds on, is that PPO relies on a separately trained value model to estimate token-level advantages $\hat{A}_t$ via Generalized Advantage Estimation (GAE) (Schulman et al., 2015), introducing significant memory overhead and a potential source of instability.

GRPO (Shao et al., 2024) was proposed to address this. It eliminates the value model by sampling a group of G responses $\{y_i\}_{i=1}^G$ for a given prompt x and computing each response’s advantage by normalizing its reward against the group’s statistics. Like PPO, however, it still optimizes the objective at the token level—the design choice that, as we argue below, becomes the root cause of instability in agentic search:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \min \left(r_{i,t}(\theta) \hat{A}_i, \text{clip} \left(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_i \right) - \beta D_{KL}(\pi_\theta || \pi_{\text{ref}}) \right], \quad (2)$$

where $r_{i,t}(\theta) = \frac{\pi_\theta(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})}$, and the advantage for every token $y_{i,t}$ in a response y_i is set to the same sequence-level value, exposing the very unit mismatch our first component removes:

$$\hat{A}_{i,t} = \hat{A}_i = \frac{R_i - \text{mean}(R)}{\text{std}(R)}, \quad (3)$$

Crucially, all tokens within a given response y_i share the same advantage $\hat{A}_i$, which is derived from the sequence-level reward.

Agentic Search as a Markov Decision Process. We model the iterative process of agentic search and reasoning as a sequential decision-making problem, formalized as a discrete-time, finite-horizon Markov Decision Process (MDP). We highlight throughout how each element of this formulation makes the case for sequence-level optimization—the first pillar of our contribution.

- **State (s_t):** At turn t , the state s_t encodes the entire interaction history: the initial question q , all previously generated thoughts and search queries, and the retrieved evidence returned by the environment. Because search results must be interpreted and integrated into subsequent decisions, the state necessarily grows with the trajectory, creating long-horizon dependencies that conventional token-level RL struggles to optimize.
- **Action (a_t):** An action is a full textual segment generated by the policy π_θ , consisting of free-form reasoning followed by a decision. The action terminates either with a `</tool_call>` token, which triggers a search, putting the results within `</tool_response>`, or with a `</answer>` token, which ends the trajectory. This structured action space forces the agent to learn not only *what* to generate but also *when* to search—an aspect that introduces significant variability in trajectory length.
- **Policy (π_θ):** The policy π_θ is the underlying LLM, generating tokens autoregressively conditioned on the state. The policy conditions on the full history, but the optimization target is calculated only on the model-generated thoughts and actions, masking out the retrieved content from the environment (Jin et al., 2025b). Optimizing this policy requires credit assignment over long sequences in which many intermediate reasoning steps do not receive direct supervision or reward, further emphasizing the need for stable sequence-level optimization.
- **Trajectory (τ):** A trajectory $\tau = (s_1, a_1, \dots, s_T, a_T)$ records all reasoning and tool interactions taken for a given question, including both model-generated actions and environment-returned search results. Because the final reward is assigned at the level of the entire trajectory, the optimization problem is fundamentally sequence-level: every early decision can influence the eventual answer correctness.
- **Reward ($R(\tau)$):** We employ a sparse terminal reward: a trajectory receives $R = 1$ if the final answer contains the ground-truth text, and $R = 0$ otherwise. As we discuss below, the sparsity of this signal is exactly what motivates the second pillar of our contribution, dynamic filtering:

$$R(\tau) = \begin{cases} 1 & \text{if } a_{\text{gold}} \subseteq a_{\text{pred}}, \\ 0 & \text{otherwise.} \end{cases} \quad (4)$$

RL with a Search Engine. Following Jin et al. (2025b), we explicitly model the search engine, denoted $\mathcal{S}$, as part of the environment. The policy LLM π_θ learns to generate trajectories by interleaving reasoning with calls to $\mathcal{S}$, and the overall goal is to find a policy that maximizes the expected reward, regularized by a KL divergence term that prevents large deviations from a reference policy π_{ref} :

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x; \mathcal{S})} [R(x, y)] - \beta D_{\text{KL}} [\pi_\theta(y|x; \mathcal{S}) || \pi_{\text{ref}}(y|x; \mathcal{S})]. \quad (5)$$

Here, $y \sim \pi_\theta(\cdot|x; \mathcal{S})$ signifies that the trajectory y is generated through a multi-step process involving both the policy’s token generation and the information returned by the search engine $\mathcal{S}$.

Motivation for DSPO. We now make explicit the two failure modes that DSPO is contributed to solve. While frameworks like Search-R1 (Jin et al., 2025b) have successfully framed agentic search as an RL problem, applying conventional algorithms like PPO or GRPO faces significant hurdles. **Failure mode 1 (instability):** the open-ended search environment exacerbates the instability of token-level optimization, whose core issue is the fundamental mismatch between the unit of sequence-level reward assignment and the unit of token-level optimization (Zheng et al., 2025); this discrepancy produces high-variance gradient estimates that accumulate over long trajectories and often culminate in policy collapse. **Failure mode 2 (vanishing signal):** the sparse binary reward means many training batches contain only successful or only unsuccessful trajectories, yielding an abnormal advantage that provides no learning

signal and drastically reduces sample efficiency (Yu et al., 2025; Liu et al., 2025). Each of DSPO’s two components is contributed as a direct countermeasure to one of these failure modes, and the next subsection presents them in that order.

3.2 DYNAMIC-FILTER SEQUENCE-LEVEL POLICY OPTIMIZATION

At the heart of our first contribution, DSPO introduces two innovations over prior methods, each mapped one-to-one onto the failure modes above: (1) it performs policy optimization at the **sequence level**, aligning the training objective with the trajectory-based reward structure to cure instability, and (2) it incorporates a **dynamic filtering** mechanism that ensures every training batch provides a high-quality, non-zero learning signal to cure the vanishing signal. The entire training process, which integrates both components, is depicted in Figure 1.

3.2.1 SEQUENCE-LEVEL POLICY OPTIMIZATION FOR ENHANCED STABILITY

This first component directly answers failure mode 1. Inspired by GSPO (Zheng et al., 2025), we replace the unstable token-level importance ratio with a theoretically grounded sequence-level counterpart. The sequence-level importance ratio $s_i(\theta)$ for a response y_i is defined as the geometric mean of its token-level ratios:

$$s_i(\theta) = \left(\frac{\pi_\theta(y_i|x)}{\pi_{\theta_{\text{old}}}(y_i|x)} \right)^{\frac{1}{|y_i|}} = \exp \left(\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \log \frac{\pi_\theta(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})} \right). \quad (6)$$

This length normalization is the crux of the stability contribution: it reduces variance and keeps $s_i(\theta)$ within a consistent numerical range regardless of sequence length, which is exactly what makes clipping stable over the long, variable-length trajectories of agentic search.

Gradient Analysis. The following analysis makes the stability contribution precise. The gradient of the token-level GRPO objective (unclipped) scales each token’s log-probability gradient by a noisy, token-specific weight $r_{i,t}(\theta)$. In contrast, the gradient of our sequence-level objective scales the average log-probability gradient of the entire sequence by a single, more stable sequence-level weight $s_i(\theta)$:

$$\nabla_\theta J_{\text{GRPO}} \propto \mathbb{E} \left[\hat{A}_i \cdot \sum_{t=1}^{|y_i|} r_{i,t}(\theta) \nabla_\theta \log \pi_\theta(y_{i,t} | \dots) \right] \quad (7)$$

$$\nabla_\theta J_{\text{DSPO}} \propto \mathbb{E} \left[\hat{A}_i \cdot s_i(\theta) \cdot \sum_{t=1}^{|y_i|} \nabla_\theta \log \pi_\theta(y_{i,t} | \dots) \right] \quad (8)$$

By applying a single, holistic correction factor to the entire trajectory, DSPO avoids the accumulation of token-level noise that plagues prior methods, delivering the fundamentally more stable training promised by Contribution 1. This stability, however, only pays off when there is a signal to learn from—which is why our second component, the dynamic filtering mechanism, guarantees a well-defined advantage $\hat{A}_i$ by constructing training batches from rollout groups that contain both successes and failures, thereby preventing wasted samples in sparse-reward environments. We turn to that component next.

3.2.2 DYNAMIC OUTCOME-BASED FILTERING FOR EFFICIENT LEARNING

This second component directly answers failure mode 2. The sparse binary nature of our reward function poses a challenge for group-based advantage estimation: if all G responses in a group are correct ($R = 1$) or all are incorrect ($R = 0$), the normalized advantage $\hat{A}_i$ becomes zero or undefined, so such batches provide no useful gradient and waste computation.

DSPO overcomes this with a dynamic filtering mechanism inspired by DAPO (Yu et al., 2025). During sampling, we retain only groups of trajectories that contain a mix of successful and unsuccessful outcomes; concretely, a group $\{y_i\}_{i=1}^G$ is used for training only if its rewards $\{R_i\}_{i=1}^G$ satisfy:

$$0 < \sum_{i=1}^G R_i < G. \quad (9)$$

Algorithm 1 Dynamic-filter Sequence-level Policy Optimization (DSPO)

```
1: Input: Initial policy  $\pi_{\theta_0}$ , fixed reference policy  $\pi_{\text{ref}}$ , prompt dataset  $\mathcal{D}$ , group size  $G$ , batch size  $B$ , search tool  $\mathcal{R}$ .
2: Initialize policy  $\pi_{\theta} \leftarrow \pi_{\theta_0}$ .
3: for each training step do
4:    $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}$ .
5:   Initialize training buffer  $\mathcal{B} \leftarrow \emptyset$ .
6:   while  $|\mathcal{B}| < B$  do
7:     Sample a prompt  $x \sim \mathcal{D}$ .
8:     Generate a group of  $G$  trajectories  $\{y_i\}_{i=1}^G$  using  $\pi_{\theta_{\text{old}}}$  and the search tool  $\mathcal{R}$ .
9:     Compute terminal rewards  $\{R_i\}_{i=1}^G = \{\text{ContainsAnswer}(y_i, y_{\text{gold}})\}_{i=1}^G$ .
10:    if  $0 < \sum_{i=1}^G R_i < G$  then ▷ Dynamic outcome-based filtering
11:      Add  $(x, \{y_i\}_{i=1}^G, \{R_i\}_{i=1}^G)$  to  $\mathcal{B}$ .
12:    end if
13:  end while
14:  for each  $(x, \{y_i\}, \{R_i\})$  in  $\mathcal{B}$  do
15:    Compute advantages  $\{\hat{A}_i\}_{i=1}^G$  via group normalization of  $\{R_i\}$ .
16:    Compute sequence-level importance ratios  $\{s_i(\theta)\}_{i=1}^G$  using Eq. 6.
17:    Compute the DSPO loss for the group using Eq. 11, applying masks to retrieved tokens.
18:  end for
19:  Update policy parameters  $\theta$  by taking a gradient step on the total loss from  $\mathcal{B}$ .
20: end for
```

This guarantees non-zero reward variance within every training group, and hence a meaningful advantage signal. The contribution of this dynamic selection is to curate a high-quality dataset for each policy update, transforming a sparse-reward problem into a dense and efficient learning signal—the efficiency half of DSPO that complements the stability half above.

3.3 THE DSPO OBJECTIVE AND TRAINING ALGORITHM

Integrating the two components yields the final DSPO objective and completes the statement of our first contribution. For each valid group from the filtered sample space $\mathcal{D}_{\text{filtered}}$, we compute the advantage $\hat{A}_i$ using group-relative normalization:

$$\hat{A}_i = \frac{R_i - \text{mean}(R)}{\text{std}(R) + \delta}, \quad (10)$$

where δ is a small constant for numerical stability. The policy π_{θ} is updated by maximizing:

$$J_{\text{DSPO}}(\theta) = \mathbb{E}_{(x, \{y_i\}) \in \mathcal{D}_{\text{filtered}}} \left[\frac{1}{G} \sum_{i=1}^G \min \left(s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) \right], \quad (11)$$

where s_i is the sequence-level importance ratio defined as the geometric mean of the token-level ratios. Crucially, during likelihood calculation we apply loss masking to all tokens retrieved from the search tool, following Jin et al. (2025b), so the model learns to utilize external knowledge for reasoning rather than simply reproduce it. Algorithm 1 details the full training process and shows precisely where each contributed component enters the loop: the dynamic filter in the sampling stage and the sequence-level ratio in the update stage.

4 EXPERIMENTS

This section provides the empirical evidence for Contributions 2 and 3, organizing every experiment around a specific claim we set out to substantiate. Concretely, we design the experiments to test whether DSPO: (1) achieves exceptional performance on challenging question-answering benchmarks (Contribution 2); (2) exhibits significantly enhanced training stability, avoiding the catastrophic collapse that plagues baseline methods (Contribution 3); and (3) derives these gains specifically from the synergistic combination of its two contributed components, as verified by ablation.

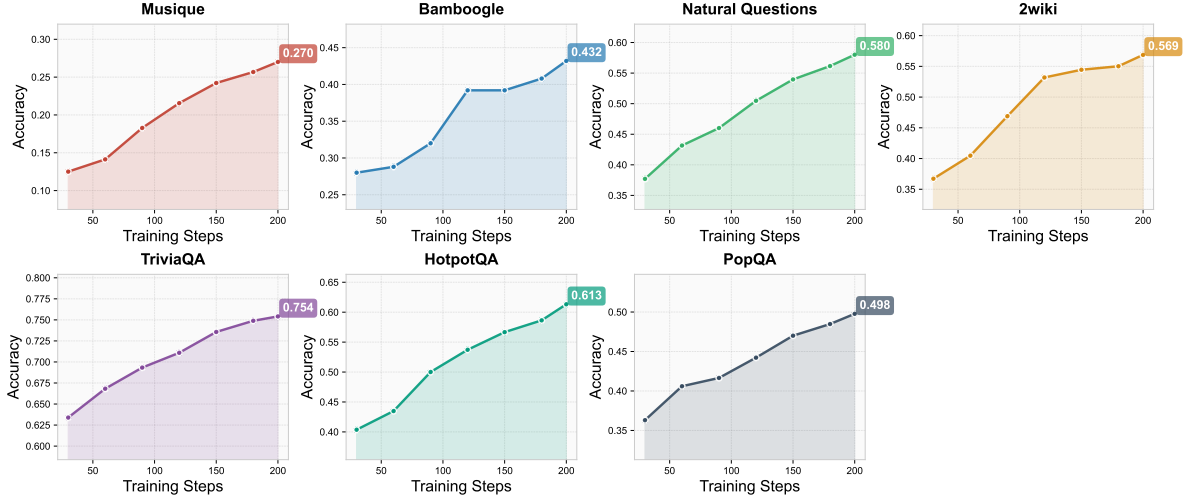


Figure 2: **Validation performance of DSPO across seven benchmarks during training.** This figure supports Contribution 3: the steady, monotonic increase in accuracy confirms that DSPO’s reward improvement translates directly into enhanced generalization rather than reward hacking, evidencing that our method learns a robust search-and-reasoning policy.

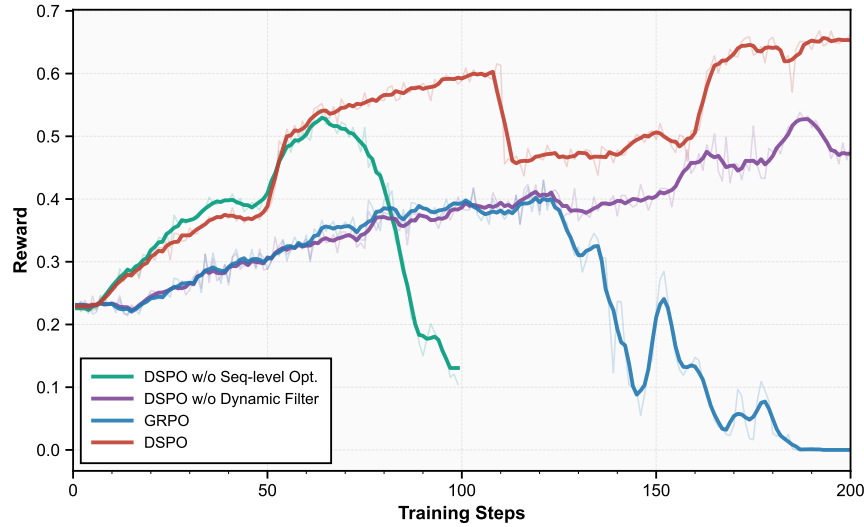


Figure 3: **Training reward dynamics of DSPO and its ablations.** This figure is the core visual evidence for Contribution 3. DSPO (red) demonstrates stable and monotonic improvement, whereas token-level variants (green, blue) suffer catastrophic policy collapse and the sequence-level variant without our filter (purple) plateaus at a suboptimal level—showing that both contributed components are needed for stable, high-reward training.

4.1 EXPERIMENTAL SETUP

To keep the comparison fair and attributable to our algorithm, we follow the Search-R1 protocol (Jin et al., 2025b) exactly. As shown in Table 1, we use a prompt template that instructs the model’s actions during the search task through `<think>`, `<tool_call>` and `<answer>` tags.

Benchmarks and Baselines. To make the evidence for Contribution 2 rigorous, our experimental design adheres to the established protocol of Search-R1 (Jin et al., 2025b). We train on a composite dataset containing the training splits of Natural Questions (NQ) (Kwiatkowski et al., 2019) and HotpotQA (Yang et al., 2018), then assess generalization

Prompt Template. Answer the given question. You must conduct reasoning inside `<think>` and `</think>`. first every time you get new information. After reasoning, if you find you lack some knowledge, you can call a search engine by `<tool_call>` query `</tool_call>` and it will return the top searched results between `<tool_response>` and `</tool_response>`. You can search as many times as your want. If you find no further external knowledge needed, you can directly provide the answer inside `<answer>` and `</answer>`, without detailed illustrations. For example, `<answer>` Beijing `</answer>`. **Question:** ...

Table 1: The prompt template used in our experiments, kept identical to Search-R1 so that any measured gains reflect our algorithm rather than prompt engineering.

on the test sets of seven diverse QA benchmarks: NQ, TriviaQA (Joshi et al., 2017), PopQA (Mallen et al., 2022), HotpotQA, 2WikiMultiHopQA (Ho et al., 2020), Musique (Trivedi et al., 2022), and Bamboogle (Press et al., 2022).

Our comparison suite is deliberately constructed to isolate each claim. For external evidence of Contribution 2, we compare against the Qwen2.5-7B and 14B models trained with PPO and GRPO from the Search-R1 framework (Jin et al., 2025b;a). To attribute the gains to our specific design choices (Contribution 3), we further include two internal ablations, each removing exactly one contributed component: (1) **DSPO w/o dynamic filter**, which is equivalent to GSPO (Zheng et al., 2025), and (2) **DSPO w/o sequence-level opt.**, which reverts to a strong token-level policy, DAPO (Yu et al., 2025).

For implementation, all our training starts from the Qwen2.5-7B-Instruct checkpoint. Our experiments are built on the VeRL framework (Sheng et al., 2025), adapting its Search-R1-like example code and scripts to our methodology. For the external comparison underpinning Contribution 2, we use the PPO and GRPO methods from the Search-R1 framework (Jin et al., 2025b;a). Because our work uses a modified reward function, we did not simply copy published numbers: we retrained these baselines under our exact experimental conditions, and these retrained models serve as our primary external benchmarks so that the reported improvements cannot be attributed to evaluation mismatch.

Implementation and Evaluation. A central design decision behind Contribution 3 is that all RL experiments deliberately employ a standard BM25 retriever. This controlled setup ensures that observed improvements are attributable to the model’s learned policy—the object of our algorithmic contribution—and not to a stronger retrieval stack. Across all methods, models are trained with a sparse, binary reward based on substring Exact Match (subEM) of the final answer, and subEM also serves as the primary evaluation metric.

4.2 MAIN RESULTS AND ABLATION STUDY

Table 2 presents the evidence for Contributions 2 and 3 in one place. Because DSPO’s value comes from the synergy of its two components, we deliberately report main results and ablations side by side: the single table juxtaposes DSPO against external state-of-the-art baselines (establishing Contribution 2) and against its own ablated variants (establishing that both components are indispensable, Contribution 3), giving reviewers a direct read on both claims at once.

Comparison with Baselines. The results in Table 2 directly confirm Contribution 2. Our DSPO-trained 7B agent reaches an average score of 0.531, a new state of the art that represents a **34.1% relative improvement** over the same-sized Search-R1 (GRPO, 7B) model. More strikingly, this 7B agent slightly edges out the much larger Search-R1 14B models under both GRPO and PPO. The headline takeaway for reviewers is that these gains come from a more effective and stable learning *algorithm*, not from added model scale—precisely the claim Contribution 2 makes.

Analysis of Ablations. The ablation results in Table 2 make the case for Contribution 3, showing that neither contributed component is optional. First, removing the dynamic filter (‘w/o dynamic filter’) causes a catastrophic drop, with the average score plummeting to 0.313: without the filter, the sequence-level objective is starved of a useful learning signal by homogeneous-reward batches. Second, ablating sequence-level optimization (‘w/o sequence-level opt.’) also degrades performance markedly, to an average of 0.406; this token-level variant beats the filter-less one but falls well short of full DSPO and, as the next section shows, is prone to catastrophic training instability. The combined message is the crux of Contribution 3: sequence-level updates supply stability while the dynamic filter supplies an efficient signal, and only their synergy recovers the full performance.

Table 2: Comprehensive comparison supporting Contributions 2 and 3: DSPO versus baselines and ablation variants on seven QA benchmarks. Baselines are the Search-R1 models (7B & 14B) trained with GRPO and PPO (Jin et al., 2025b;a); ablations remove one contributed component each (‘w/o dynamic filter’ and ‘w/o seq-level opt.’). Original EM scores from Search-R1 are shown in parentheses; for evaluation consistency we retrained and re-evaluated all baselines under our adjusted rewards. Best results are in **bold** and second-best are underlined, so the reader can see at a glance that our 7B DSPO leads or ties the field.

Dataset	Search-R1				DSPO & Ablations (Ours, 7B)		
	GRPO (7B)	PPO (7B)	GRPO (14B)	PPO (14B)	w/o dyn. filter	w/o seq-lvl opt.	DSPO
NQ	0.423 (0.429)	(0.393)	<u>0.535</u> (0.482)	(0.424)	0.363	0.470	0.580
TriviaQA	0.658 (0.623)	(0.610)	0.760 (0.667)	(0.660)	0.515	0.695	<u>0.754</u>
PopQA	0.395 (0.427)	(0.397)	<u>0.477</u> (0.434)	(0.442)	0.277	0.430	0.498
HotpotQA	0.401 (0.386)	(0.370)	<u>0.563</u> (0.429)	(0.436)	0.330	0.438	0.613
2WikiMultiHopQA	0.357 (0.414)	(0.346)	0.611 (0.424)	(0.379)	0.285	0.398	<u>0.569</u>
Musique	0.122 (0.162)	(0.146)	<u>0.260</u> (0.191)	(0.210)	0.105	0.133	0.270
Bamboogle	0.280 (0.400)	(0.368)	0.504 (0.492)	(0.480)	0.288	0.280	<u>0.432</u>
Average	0.377 (0.396)	(0.385)	<u>0.530</u> (0.446)	(0.433)	0.313	0.406	0.531

Beyond the quantitative metrics, we observe a qualitative dividend that reinforces our second contribution: DSPO elicits sophisticated search behaviors—recognizing irrelevant results, reformulating queries, and verifying answers across multiple turns (see Appendix A for detailed trajectory examples). Notably, all of these behaviors emerge from pure RL training through DSPO, with no supervised demonstrations.

4.3 ANALYSIS OF TRAINING DYNAMICS

We now turn to the direct evidence for Contribution 3 (stability), analyzing the training reward dynamics of DSPO, its ablations, and key baselines. Figure 3 visualizes these dynamics and, read alongside the ablation table, closes the loop on our stability claim.

The curves separate cleanly along the two components we contribute. **DSPO (red)** exhibits a smooth, monotonic ascent to the highest reward, empirically confirming the stability afforded by its sequence-level objective. In stark contrast, the **token-level methods**—DSPO w/o Seq-level Opt. (green) and vanilla GRPO (blue)—suffer catastrophic policy collapse early in training, their rewards plummeting after a brief rise; this is the high-variance, token-level instability that our first component removes. Meanwhile, **DSPO w/o Dynamic Filter** (purple) uses sequence-level updates but lacks an efficient signal, so it stays stable yet plateaus well below DSPO; this is exactly the deficiency our second component fixes. Together these dynamics show that neither component alone suffices—the synergy of sequence-level stability and dynamic filtering is what makes DSPO robust and effective, substantiating Contribution 3.

To rule out the obvious reviewer objection—that higher training reward could reflect reward hacking rather than real skill—we track validation performance throughout training. As Figure 2 shows, DSPO’s validation accuracy on NQ, HotpotQA, and other diverse benchmarks rises consistently and mirrors its stable reward curve. This tight correlation confirms that the agent learns a genuinely generalizable search-and-reasoning policy, closing the argument for Contributions 2 and 3.

4.4 SCALABILITY AND GENERALIZATION ANALYSIS

To probe how far our contributions generalize, we extend the evaluation along two axes—model scale and task domain—each stress-testing whether the benefits of DSPO are specific to the 7B QA setting or hold more broadly.

Scalability to Larger Models. We first ask whether DSPO’s stability benefits persist at larger parameter scales by training Qwen2.5-14B-Instruct. As Table 3 details, they do: the DSPO-trained 14B model reaches an average accuracy of 60.6%, outperforming the strong GRPO-14B baseline (53.0%) by a relative margin of **14.3%**. This shows that our algorithmic contribution effectively leverages added model capacity, with the advantage over standard baselines widening rather than washing out at scale.

Table 3: Scalability analysis on Qwen2.5-14B-Instruct, extending Contribution 2 to a larger backbone. Best results are in **bold**; the final column reports DSPO-14B’s relative gain over the GRPO-14B baseline.

Dataset	Instruct (14B)	GRPO (14B)	DSPO (7B)	DSPO (14B)	Gain
NQ	0.345	0.535	0.580	0.629	+17.6%
HotpotQA	0.407	0.563	0.613	0.665	+18.1%
2WikiMQA	0.332	0.611	0.569	0.699	+14.4%
Bamboogle	0.328	0.504	0.432	0.544	+7.9%
PopQA	0.364	0.477	0.498	0.545	+14.3%
TriviaQA	0.643	0.760	0.754	0.802	+5.5%
Musique	0.151	0.260	0.270	0.361	+38.8%
Average	0.367	0.530	0.531	0.606	+14.3%

Generalization to Mathematical Reasoning. Finally, we test whether the contribution reaches beyond search by applying DSPO to single-turn mathematical reasoning with the Qwen2.5 and Qwen3 model families. As Table 4 reports on Math500 and Olympiad-Bench, DSPO consistently surpasses GRPO at both 7B and 4B sizes. This is early but concrete evidence that the principles behind our contribution transfer to general reasoning domains, foreshadowing the broader impact we discuss in the conclusion.

Table 4: Generalization to mathematical reasoning, evidence that our contribution extends beyond agentic search. Best results are in **bold**; the final column reports DSPO’s relative gain over GRPO.

Model	Benchmark	Steps	GRPO	DSPO	Gain
Qwen2.5-Math-7B	Math500	200	0.772	0.798	+2.6%
Qwen3-4B	Olympiad-Bench	100	0.728	0.755	+2.7%

5 CONCLUSION

We conclude by restating what this paper contributes and what the evidence establishes. We tackled the critical instability and sample-inefficiency issues that plague RL for autonomous LLM search agents. Our **first contribution** is Dynamic-filter Sequence-level Policy Optimization (DSPO), an improved algorithm that ensures robust training through two components: sequence-level optimization to prevent catastrophic policy collapse, and a dynamic outcome-based filter that turns sparse rewards into a consistently effective learning signal. Our **second and third contributions** are empirical: across a suite of challenging QA benchmarks, DSPO delivers substantial performance gains (a 34.1% relative improvement over a comparable 7B baseline, and 7B results that match its 14B counterpart), while exhibiting markedly superior training stability under a controlled BM25 setup.

The broader significance of these contributions is that DSPO enables robust training from environmental feedback alone, establishing a practical and efficient blueprint for building capable LLM agents without costly expert data. On this stable foundation, future work can confidently integrate advanced retrievers or extend DSPO to complex, multi-tool tasks. Moreover, because sparse rewards and unstable policy gradients are not unique to search—and as our mathematical-reasoning results already hint—we expect DSPO’s principles to yield similar gains in domains such as mathematics and code generation, a promising direction for further validation. Ultimately, we believe the two core tenets underlying our contribution—matching the optimization unit to the reward signal, and guaranteeing signal density—will be instrumental in developing the next generation of autonomous AI.

REFERENCES

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

- Tuhin Chakrabarty, Vishakh Padmakumar, Faeze Brahman, and Smaranda Muresan. Creativity support in the age of large language models: An empirical study involving professional writers. In *Proceedings of the 16th Conference on Creativity & Cognition*, pp. 132–155, 2024.
- Mingyang Chen, Tianpeng Li, Haoze Sun, Yijie Zhou, Chenzheng Zhu, Haofen Wang, Jeff Z Pan, Wen Zhang, Huajun Chen, Fan Yang, et al. Learning to reason with search for llms via reinforcement learning. *arXiv preprint arXiv:2503.19470*, 2025.
- Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.
- Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V Le, Sergey Levine, and Yi Ma. Sft memorizes, rl generalizes: A comparative study of foundation model post-training. *arXiv preprint arXiv:2501.17161*, 2025.
- Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, et al. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2(1), 2023.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. *arXiv preprint arXiv:2011.01060*, 2020.
- Bowen Jin, Jinsung Yoon, Priyanka Kargupta, Sercan O Arik, and Jiawei Han. An empirical study on reinforcement learning for reasoning-search interleaved llm agents. *arXiv preprint arXiv:2505.15117*, 2025a.
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-rl: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025b.
- Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *arXiv preprint arXiv:1705.03551*, 2017.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick SH Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *EMNLP (1)*, pp. 6769–6781, 2020.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding rl-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Hannaneh Hajishirzi, and Daniel Khashabi. When not to trust language models: Investigating effectiveness and limitations of parametric and non-parametric memories. *arXiv preprint arXiv:2212.10511*, 7, 2022.
- Guillermo Marco, Julio Gonzalo, Ramón del Castillo, and María Teresa Mateo Girona. Pron vs prompt: Can large language models already challenge a world-class fiction author at creative text writing? *arXiv preprint arXiv:2407.01119*, 2024.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*, 2022.

- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 1279–1297, 2025.
- Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592*, 2025.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Trieu H Trinh, Yuhuai Wu, Quoc V Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- Junde Wu, Jiayuan Zhu, Yuyuan Liu, Min Xu, and Yueming Jin. Agentic reasoning: A streamlined framework for enhancing llm reasoning with agentic tools. *arXiv preprint arXiv:2502.04644*, 2025.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600*, 2018.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2), 2023.
- Yuzhong Zhao, Yue Liu, Junpeng Liu, Jingye Chen, Xun Wu, Yaru Hao, Tengchao Lv, Shaohan Huang, Lei Cui, Qixiang Ye, et al. Geometric-mean policy optimization. *arXiv preprint arXiv:2507.20673*, 2025.

Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.

Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, et al. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 5673–5684, 2023.

A APPENDIX

To give qualitative insight into the search strategies our contribution elicits, Table A.1 presents representative trajectories from our model. These examples concretely illustrate the emergent behaviors cited in the main results—the model (1) reformulates queries when initial searches return irrelevant results, (2) verifies information through follow-up searches, and (3) extracts comprehensive answers from search results—all learned through DSPO’s pure-RL training.

Table A.1: Representative trajectories that make the emergent capabilities behind Contribution 2 tangible. The DSPO-trained model exhibits sophisticated behaviors including (a) recognition of irrelevant results, (b) query reformulation, and (c) verification strategies—none of them supervised, all emerging from RL.

Example 1: Presidential Succession (<i>Query Refinement after Failure</i>)	
Question	Who is next in line if something happens to the president?
Gold	vice president
Turn 1	<code>search("who is next in line if something happens to the president")</code> → Retrieved: Documents about novels "Something Happened" and "Mosquitoes Buzz" Agent: "The queries were not directly about the line of succession... The relevant information I need is not present in the search results. Let's try searching again with a more specific query ."
Turn 2	<code>search("line of succession for the president")</code> → Retrieved: Philippine and US presidential succession information Agent: "Based on the search results... the next person in line would be the Vice President."
Answer	✓ The Vice President
Example 2: University Identification (<i>Information Verification</i>)	
Question	Which American University is situated in Ithaca, New York?
Gold	Cornell University
Turn 1	<code>search(["American University in Ithaca", "University in Ithaca NY"])</code> → Retrieved: "Ithaca is home to Cornell University, an Ivy League school..." Agent: "Ithaca, New York is home to <u>Cornell University</u> , which is an Ivy League school. However... it is highly likely that the university in question is Cornell. Let me confirm this. "
Turn 2	<code>search(["Cornell University location", "Cornell University New York"])</code> → Retrieved: Cornell history confirming establishment in Ithaca Agent: "Cornell University is indeed located in Ithaca, New York."
Answer	✓ Cornell University
Example 3: Biographical Information (<i>Direct Extraction</i>)	
Question	What is John Andrew Martin’s occupation?
Gold	lawyer, attorney, politician
Turn 1	<code>search(["John Andrew Martin occupation", "John Andrew Martin job"])</code> → Retrieved: "American journalist, attorney, soldier, and politician..." Agent: "John Andrew Martin was an American <u>journalist, attorney, soldier, and politician</u> . He represented Colorado in the U.S. House of Representatives."
Answer	✓ Journalist, attorney, soldier, and politician